

Dynamics 365 CRM Interview Questions with Answers and Examples

Tech Masala

Table of Contents

A. Business Rules & Client-Side Customization

1. Client-side form validations used
2. Business rule scope
3. Difference between Entity and All Forms
4. What triggers first – business rule or JavaScript?
5. Business rule triggers on database operations
6. Filtering records on a lookup field
7. Register custom filter for a lookup
8. Trigger JS on BPF stage change
9. UseStrict in JavaScript
10. Promises vs Callbacks

B. Plugins

11. Plugin execution stages
12. PreValidation vs PreOperation
13. Shared variable usage across stages
14. Plugin impersonation
15. PreValidation scenario: create account from email check
16. Async plugin to create opportunity – frontend error?
17. Sync + Async plugin: what gets created?

C. Entity & Data Model

18. Relationships in entities
19. POA records for owner/access teams

D. Solution Management

20. Managed vs Unmanaged solutions
21. Modify managed form from unmanaged solution?

E. API Integration

- 22. Integrate CRM APIs using events
- 23. Azure app authentication with CRM API
- 24. Authenticate External API with Power Pages
- 25. Azure Service Bus usage
- 26. Xrm.WebAPI to create record
- 27. Fetch more than 5000 records

F. Power Automate (Flow)

- 28. Exception handling in Power Automate
- 29. Flow trigger on firstname change and country = India

G. Data Queries

- 30. Expand query
- 31. QueryExpression for names starting with 'A'
- 32. FetchXML with distinct

H. Business Process Flows

- 33. What is a Business Process Flow?

I. Programming Concepts

- 34. Abstraction vs Inheritance

J. Routing & Unified Interface

- 35. Unified Routing
- 36. Routing based on agent availability
- 37. Agent Profiles

K. SLA and Case Management

- 38. What is SLA?
- 39. Custom timer in SLA
- 40. Auto-case creation from activity
- 41. SLA reset – which SLA starts?
- 42. SLA KPI creation
- 43. Action on SLA success – auto or custom?
- 44. SLA vs SLA KPI vs SLA Item

L. Security

- 45. SLA assignment when multiple SLAs exist
- 46. Read-only access to Account, full access to Contact
- 47. Sales manager accessing subordinates' data only

48. Restrict confidential field to HR only
49. Access for project team without ownership
50. User loses access after BU change – fix?
51. Share opportunity across BUs without ownership change
52. Update-only access to owned records
53. Prevent delete but allow create/edit
54. Team-based shared lead ownership
55. Manager access to direct report records
56. Security models in Dynamics CRM
57. Access levels in security roles
58. Multiple roles – how access is calculated
59. Owner teams vs Access teams
60. Field-level security
61. Effects of BU removal on access
62. Purpose of Business Units
63. How Hierarchy Security works
64. Security roles vs Privileges
65. Auditing access/update in CRM

A. Business Rules & Client-Side Customization

1. What are the client-side form validations that you have used?

Client-side form validations in Dynamics 365 CRM are implemented using JavaScript or Business Rules to enforce data integrity on forms. Examples include:

- Checking for required fields.
- Validating email formats using regex.
- Ensuring numerical fields fall within a specific range.
- Using Business Rules to show/hide fields or set field values based on conditions.

Example:

```
function validateEmail(executionContext) {  
    var formContext = executionContext.getFormContext();  
    var email = formContext.getAttribute("emailaddress1").getValue();  
    var regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
    if (!regex.test(email)) {  
        formContext.getControl("emailaddress1").setNotification("Invalid email format.");  
    } else {  
        formContext.getControl("emailaddress1").clearNotification();  
    }  
}
```

2. What is a business rule scope?

Business Rule scope defines where the rule is applied:

- **Entity:** Applies to all forms and server-side logic for the entity.
- **All Forms:** Applies to all forms of the entity but only client-side.
- **Specific Form:** Applies only to a specific form (e.g., Main Form).

Example: A rule to make a field required can be set to "Entity" to enforce it across all forms and server-side.

3. What is the difference between Entity and All Forms?

Entity scope applies the rule to all forms and server-side operations (e.g., during data import or API updates). **All Forms** scope applies only to client-side interactions across all forms, not server-side.

4. What will trigger first - business rule or JavaScript in a form?

Business Rules trigger before JavaScript event handlers on form events like OnLoad or OnChange. Business Rules are executed as part of the form's rendering process, while JavaScript is triggered afterward based on registered events.

5. When will a business rule trigger for any database operations?

Business Rules with "Entity" scope trigger during database operations (e.g., create/update via API or workflows). Client-side scoped rules (All Forms or Specific Form) do not trigger for database operations.

6. How do you filter the records on the lookup field?

Lookup field filtering can be done using `addCustomFilter` or `addPreSearch` in JavaScript to apply FetchXML filters dynamically. **Example:**

```
function filterLookup(executionContext) {
    var formContext = executionContext.getFormContext();
    var fetchXml = "<filter><condition attribute='statecode' operator='eq' value='0' /></filter>";
    formContext.getControl("parentaccountid").addCustomFilter(fetchXml, "account");
}
```

7. How to register event for custom filter for a lookup?

Use addPreSearch to register a custom filter function for a lookup field. **Example:**

```
function onFormLoad(executionContext) {
    var formContext = executionContext.getFormContext();
    formContext.getControl("parentaccountid").addPreSearch(
        filterLookup);
}
```

8. How to trigger a JS On-stage change of a BPF?

Register a JavaScript function on the OnStageChange event of a Business Process Flow (BPF). **Example:**

```
function onStageChange(executionContext) {
    var formContext = executionContext.getFormContext();
    var activeStage = formContext.data.process.getActiveStage();
    if (activeStage.getName() === "Qualify") {
        Xrm.Utility.alertDialog("Stage changed to Qualify!");
    }
}
```

Register via form properties: `formContext.data.process.addOnStageChange(onStageChange)`

9. What is UseStrict used for in JavaScript?

"use strict"; enables strict mode in JavaScript, enforcing stricter parsing and error handling, preventing undeclared variables, and improving code quality. **Example:**

```
"use strict";
function myFunction() {
    x = 10; // Error: x is not declared
}
```

10. Promise functions and callback?

Promises handle asynchronous operations, providing then, catch, and finally methods. Callbacks are functions passed as arguments to execute after an operation completes. **Example (Promise):**

```
function getRecord() {
    return new Promise((resolve, reject) => {
        Xrm.WebApi.retrieveRecord("account", "guid", "?$select=name").
            then(
                result => resolve(result.name),
                error => reject(error)
            );
    });
}
getRecord().then(name => console.log(name)).catch(error => console.
    error(error));
```

B. Plugins

11. What are the plugin execution stages in CRM?

Plugins execute in stages:

- **PreValidation (10):** Before validation, outside transaction.
- **PreOperation (20):** Before database operation, inside transaction.
- **PostOperation (40):** After database operation, inside transaction.

12. What is the difference between PreValidation and PreOperation stage?

PreValidation runs before data validation, outside the transaction, and can be used to cancel operations. **PreOperation** runs after validation, inside the transaction, and is used for data modification before saving.

13. What is shared variable? Can we pass variable value from PreValidation to PreOperation?

Shared variables are key-value pairs in `IPluginExecutionContext.SharedVariables` to pass data between plugin steps. Yes, variables can be passed from PreValidation to PreOperation. **Example:**

```
public void Execute(IServiceProvider serviceProvider) {
    var context = (IPluginExecutionContext)serviceProvider.GetService(
        typeof(IPluginExecutionContext));
    if (context.Stage == 10) { // PreValidation
        context.SharedVariables["MyKey"] = "MyValue";
    } else if (context.Stage == 20) { // PreOperation
        var value = context.SharedVariables["MyKey"];
    }
}
```

14. How do you impersonate in a Plugin?

Impersonation is done by setting the `InitiatingUserId` in the plugin context or using `IOrganizationServiceFactory.CreateOrganizationService` with a specific user ID. **Example:**

```
var serviceFactory = (IOrganizationServiceFactory)serviceProvider.
    GetService(typeof(IOrganizationServiceFactory));
var impersonatedService = serviceFactory.CreateOrganizationService(new
    Guid("impersonated-user-guid"));
```

15.

using plugin in case of a PreValidation scenario?**Scenario: If we check whether an email Id ends with .com and based on that it creates an account, then how does it work**

using plugin in case of a PreValidation scenario?

In PreValidation, validate the email and create an account if valid. Example:

```
public void Execute(IServiceProvider serviceProvider) {
    var context = (IPluginExecutionContext)serviceProvider.GetService(
        typeof(IPluginExecutionContext));
    var serviceFactory = (IOrganizationServiceFactory)serviceProvider.
        GetService(typeof(IOrganizationServiceFactory));
    var service = serviceFactory.CreateOrganizationService(context.
        UserId);
    var entity = (Entity)context.InputParameters["Target"];
```

```

var email = entity.GetAttributeValue<string>("emailaddress1");
if (email.EndsWith(".com")) {
    var account = new Entity("account") { ["name"] = "New Account"
    };
    service.Create(account);
} else {
    throw new InvalidPluginExecutionException("Email must end with
        .com");
}
}

```

16. **Scenario: If we need to create an opportunity in an account through plugin in an asynchronous mode, will it create an account or generate an error on the frontend side?**

In asynchronous mode, the plugin runs in the background. If the account exists, the opportunity is created without frontend errors. If the account creation fails (e.g., due to invalid data), the async plugin logs the error in the system job, not on the frontend.

17. **On create of account, we have a sync plugin sync plugin will create contact record and throw an exception to user, and async plugin async plugin will create an opportunity and throw exception. Account will be created or not? And opportunity will be created or not?**

The synchronous plugin runs first. If it throws an exception, the account creation fails, and the async plugin does not execute. Thus, neither the account nor the opportunity is created. If the sync plugin succeeds, the account is created, and the async plugin attempts to create the opportunity, logging any errors in the system job.

C. Entity & Data Model

18. **What are the relationships in an entity?**

Relationships in Dynamics 365 include:

- **1:N (One-to-Many):** One record relates to many (e.g., one account to many contacts).
- **N:1 (Many-to-One):** Many records relate to one (e.g., many contacts to one account).
- **N:N (Many-to-Many):** Multiple records relate to multiple records (e.g., contacts to marketing lists).

19. **How many records are created in a POA table for owner teams and access teams when records are created within?**

The Principal Object Access (POA) table stores sharing records. For **owner teams**, one POA record is created per shared record per team. For **access teams**, one POA record is created per user in the access team per shared record.

D. Solution Management

20. What are the different solutions during deployment? Explain the difference?

- **Managed Solution:** Read-only, controlled by the publisher, used in production.
- **Unmanaged Solution:** Editable, used for development and customization.

Managed solutions can't be modified directly; unmanaged solutions allow full customization.

21. **Scenario: If I need to modify a CRM form and I have the form in a managed solution, and on top of it I have created an unmanaged solution, can I update the form or not?**

Yes, you can modify the form in the unmanaged solution. Unmanaged solutions override managed solution components, allowing form customizations without altering the managed solution.

E. API Integration

22. **What are the ways to integrate CRM APIs with external system/apps using an event? How have you used it in your project?**

Integration methods:

- **Webhooks:** Trigger external APIs on CRM events.
- **Plugins:** Call external APIs from plugins.
- **Power Automate:** Use flows to connect CRM events to external systems.

Example: Used Power Automate to call a REST API when a case is created, sending data to an external ticketing system.

23. **How do you authenticate the Azure apps with CRM API?**

Use Azure AD with OAuth 2.0. Register the app in Azure AD, grant permissions to Dynamics 365, and use client credentials or authorization code flow to obtain an access token. **Example:**

```
const { Client } = require("msal-node");
const client = new Client({
  auth: {
    clientId: "your-app-id",
    authority: "https://login.microsoftonline.com/your-tenant-id",
    clientSecret: "your-secret"
  }
});
client.acquireTokenByClientCredential({
  scopes: ["https://your-org.crm.dynamics.com/.default"]
}).then(token => console.log(token.accessToken));
```

24. **How to authenticate External API with Power Pages?**

Use Azure AD, OAuth, or API keys. Configure the external API in Power Pages site settings and use Power Automate or JavaScript to authenticate. **Example:** Use OAuth token in a web template to call an external API.

25. **Any experience on Azure Service Bus?**

Azure Service Bus can be used to queue messages from Dynamics 365 for asynchronous processing. Example: Register a service endpoint in CRM to send messages to Azure Service Bus when a record is updated.

26. **Code for Xrm WebAPI to create a record?**

Example:

```
function createAccount() {
  var account = {
    "name": "Sample Account",
    "industrycode": 1
  };
  Xrm.WebApi.createRecord("account", account).then(
    result => console.log("Record created: " + result.id),
    error => console.error(error.message)
  );
}
```

27. **How do you fetch more than 5000 records?**

Use paging with `odata.nextLink` in Web API or paging-cookie in FetchXML to

retrieve records beyond 5000. **Example:**

```
function fetchMoreRecords() {  
    var fetchXml = "<fetch><entity name='account'><attribute name='  
        name' /></entity></fetch>";  
    Xrm.WebApi.online.executeFetchXmlAll("account", fetchXml).then(  
        results => console.log(results),  
        error => console.error(error)  
    );  
}
```

F. Power Automate (Flow)

28. Exception handling in Power Automates.

Use Scope to group actions and configure run after settings to handle failures. Add a Compose or Set Variable action to log errors. **Example:** Create a Scope for actions, add a parallel branch for error handling, and use Configure run after to execute on failure.

29. How to configure a flow that will trigger when first name changes and country = India? Can we add expand query for sub-region or not?

Use a trigger condition like `@equals(triggerOutputs()?['body/country'], 'India')` and a filter on firstname change. Expand queries are supported in Power Automate to retrieve related records. **Example:** In a flow, use List Records with `$expand=subregion` to fetch related data.

G. Data Queries (FetchXML / QueryExpression / Expand)

30. What is expand query?

Expand query retrieves related records in a single query using \$expand in Web API or linked-entity in FetchXML. **Example:**

```
Xrm.WebApi.retrieveMultipleRecords("account", "?$select=name&$expand=contact_customer_accounts($select=fullname)").then(
    results => console.log(results)
);
```

31. Code for QueryExpression where name begins with 'A'?

Example:

```
QueryExpression query = new QueryExpression("account") {
    ColumnSet = new ColumnSet("name"),
    Criteria = new FilterExpression() {
        Conditions = { new ConditionExpression("name",
            ConditionOperator.BeginsWith, "A") }
    }
};
var results = service.RetrieveMultiple(query);
```

32. In FetchXML, do we use distinct?

Yes, FetchXML supports distinct='true' to remove duplicate records. **Example:**

```
<fetch distinct='true'>
  <entity name='account'>
    <attribute name='name' />
  </entity>
</fetch>
```

H. Business Process Flows

33. Business Process Flows?

BPFs guide users through a defined process with stages and steps. They can be customized to include conditions, actions, and JavaScript events. **Example:** A BPF for lead qualification with stages: Qualify, Develop, Propose.

I. Programming Concepts (Relevant for Dynamics)

34. Difference between Abstraction and Inheritance?

Abstraction hides implementation details, exposing only necessary features (e.g., interfaces). **Inheritance** allows a class to inherit properties and methods from another class. **Example (C#):**

```
public abstract class Animal {
    public abstract void MakeSound();
}
public class Dog : Animal {
    public override void MakeSound() { Console.WriteLine("Bark"); }
}
```

J. Routing & Unified Interface

35. Unified Routing?

Unified Routing distributes work items (e.g., cases, leads) to agents based on rules, skills, or availability in Omnichannel for Customer Service.

36. How to configure routing so that the case is assigned to the agent whose presence is available?

Configure a routing rule in Omnichannel to check agent presence using the Presence condition and assign cases to available agents.

37. What are Agent Profiles used for?

Agent Profiles define skills, capacity, and roles for agents in Omnichannel to match work items during routing.

K. SLA and Case Management

38. What is SLA?

Service Level Agreements (SLAs) define response and resolution time targets for cases, ensuring timely customer service.

39. How to create custom timer in SLA?

Create a timer in an SLA item with a specific duration and condition. **Example:** Set a timer to resolve a case within 4 hours using SLA configuration in Dynamics 365.

40. How to create automatic Case from activity, how to configure that?

Use a workflow or Power Automate flow to create a case from an activity (e.g., email). Configure a trigger on activity creation and use Create Record action. **Example:** In Power Automate, trigger on email creation, check conditions, and create a case.

41. We have two SLAs, if the first gets executed and we reset it, will it start from first SLA or second?

Resetting an SLA restarts the first applicable SLA based on the defined order and conditions.

42. How is SLA KPI created?

SLA KPIs are created in the SLA configuration, defining metrics like First Response By or Resolve By. **Example:** Create an SLA KPI for case resolution within 8 hours.

43. If SLA succeeds and we need to perform an action, how do we do it? Will the code be auto-created or do we write it?

Use a workflow or Power Automate flow triggered on SLA success to perform actions. The code is not auto-created; it must be written. **Example:** Create a flow to send a notification on SLA success.

44. Explain in detail about SLA, SLA KPI, and SLA item?

- **SLA:** Defines service targets for cases.
- **SLA KPI:** Metrics to track SLA performance (e.g., time to first response).
- **SLA Item:** Specific conditions and timers within an SLA (e.g., resolve case within 4 hours if priority is high).

L. Security

45. **How would you assign SLA to a case if there are multiple SLAs?**
Assign SLAs based on priority or conditions in the SLA configuration. The first matching SLA is applied.
46. **A user needs read-only access to the Accounts entity but full access to Contacts. How will you configure the security role?**
Create a security role with Read privilege for Accounts and Read, Write, Create, etc., for Contacts at the appropriate access level (e.g., User or Business Unit).
47. **You want a sales manager to view and update records of their subordinates, but not those from other business units. How would you implement this?**
Enable **Hierarchy Security** and configure the manager's security role with Read and Write privileges at the Parent:Child Business Units level.
48. **A confidential field like Annual Salary must be hidden from all users except HR managers. How would you configure this?**
Use **Field-Level Security**. Create a field security profile, add the Annual Salary field, and assign it to HR managers with Read and Update permissions.
49. **A team is collaborating on a project, and all members need to access specific records without owning them. How would you achieve this?**
Use **Access Teams**. Create an access team, share specific records with the team, and grant appropriate privileges.
50. **A user was moved to a new Business Unit and lost access to records. What steps would you take to restore access?**
- Verify the user's security role in the new Business Unit.
 - Share records explicitly or assign the user to an access team.
 - Check if hierarchy security or team membership needs adjustment.
51. **How would you give access to a single opportunity record to another user from a different business unit without changing ownership?**
Use **Manual Sharing**. Share the opportunity record with the user, granting specific privileges (e.g., Read, Write).
52. **You need to allow users to update only the records they own, and not others, even in the same business unit. How will you handle this?**
Configure the security role with Read and Write privileges at the User level for the entity.
53. **How would you prevent users from deleting records but still allow them to create and edit records in a specific entity?**
Set the security role's Delete privilege to None while granting Create and Write privileges at the desired level.
54. **How do you configure security so that a team of users can own and manage shared leads together?**
Create an **Owner Team**, assign a security role with appropriate privileges, and assign leads to the team.
55. **Your client asks to enforce a rule where managers can access all records created by their direct reports, regardless of business unit. How would you im-**

plement it?

Enable **Hierarchy Security** with the Manager model and configure the security role with Read and Write at the Organization level for direct reports.

56. **What are the different security models available in Dynamics 365 CRM?**

- **Role-Based Security:** Privileges defined by security roles.
- **Record-Based Security:** Access control via ownership, sharing, or teams.
- **Field-Level Security:** Restrict access to specific fields.
- **Hierarchy Security:** Manager or position-based access.

57. **What are the access levels in a security role?**

- **None:** No access.
- **User:** Access to user-owned records.
- **Business Unit:** Access to records in the user's business unit.
- **Parent:Child Business Units:** Access to records in the user's and child business units.
- **Organization:** Access to all records.

58. **Can a user have multiple security roles? How is access calculated?**

Yes, a user can have multiple security roles. Access is calculated by combining the highest privileges from all roles (union of privileges).

59. **What is the difference between owner teams and access teams?**

Owner Teams can own records and have security roles assigned. **Access Teams** are used for sharing records and do not own records.

60. **What is field-level security in Dynamics 365 CRM?**

Field-Level Security restricts access to specific fields, allowing only authorized users to read, update, or create field values.

61. **What happens when a user is removed from a Business Unit?**

The user loses access to records tied to the old business unit unless explicitly shared or accessible via teams or hierarchy security.

62. **What is the purpose of a business unit in the security model?**

Business Units segment users and records, controlling access based on organizational structure.

63. **How does hierarchy security work in Dynamics 365 CRM?**

Hierarchy Security grants managers access to records of their direct reports or based on position hierarchy, regardless of business unit.

64. **What is the difference between security roles and privileges?**

Security Roles are collections of **Privileges** (e.g., Create, Read, Write) that define what actions a user can perform on entities.

65. **How can you audit who accessed or updated a record in Dynamics 365 CRM?**

Enable auditing in the entity settings and review audit logs in the Audit History or Audit Summary view.

☒ Security-Related Questions

1. What is a Root Business Unit?

- The **Root Business Unit** is the **top-level business unit** in Dynamics 365 CRM and is automatically created when an organization is provisioned.
- It cannot be deleted or renamed.
- All users and teams must belong to some business unit (directly or via hierarchy).
- Security roles are defined per business unit and control access across the CRM hierarchy.

Example:

If the root BU is “Contoso HQ,” you can have child BUs like “Sales India” or “Marketing US” under it. Assigning a security role at the child BU will not allow access to records in other BUs unless privilege level is set to **Organization**.

2. What is the Difference Between Append and Append To Privileges?

- **Append:** Allows a record to be related to another record.
- **Append To:** Allows a record to have other records related to it.

Example:

Attaching a **Contact** to an **Account**:

- Contact needs **Append To**
- Account needs **Append**

This means: "Contact is appended to Account."

3. What is Field Level Security (FLS)?

- FLS restricts **access to individual fields** based on a user's security profile.
- You can control:
 - **Read**
 - **Create**
 - **Update**

Example:

The Annual Revenue field on an Account entity might only be visible/editable to senior management, even if the sales team can view the rest of the record.

☒ CRM Lifecycle, Plugins, and API Questions

4. What is a Customer Journey?

- Found in **Dynamics 365 Marketing**.
- A Customer Journey defines **automated steps** (emails, events, form responses) a customer goes through.
- Used for **targeted marketing** and lead nurturing.

Example:

A visitor submits a form → receives email → added to a segment → triggers a follow-up → notifies sales.

5. At What Stage Can Products Be Associated in the Lifecycle? Do Lead Entities Have Relationships with Products?

- Leads do not have direct relationships with Products.
- Product association begins at **Opportunity stage**.

Example:

Qualify a Lead → Opportunity is created → Add **Opportunity Products** (linked to Products catalog).

6. Can the Create Message Work in Pre-Operation? No

GUID Is Present — How to Update the Record?

- Yes, you can use **Pre-Operation Create** plugin to modify fields before save.
- Although the record does not have a **GUID yet**, you can still modify the **Target** entity.

Example:

csharp

CopyEdit

```
Entity target = (Entity)context.InputParameters["Target"]; target["new_status"] =  
"AutoGenerated"; // This updates before saving
```

7. What If We Pass Null or String.Empty Instead of context.UserId?

- context.UserId contains the **initiating user's GUID**.
- Passing Guid.Empty or null can lead to:
 - Security errors
 - NullReferenceExceptions
 -

Example:

csharp

CopyEdit

```
Guid userId = context.UserId; if (userId == Guid.Empty) { throw new  
InvalidPluginExecutionException("User ID missing."); }
```

8. What Is the Default Execution Order of Async Plugin?

- Default order is 1, unless specified.
- Async plugins run **post-transaction**, but **sequence matters** within the same message and stage.

You can set execution order in Plugin Registration Tool.

9. At Execution Order 0, What Will Run First — Async Plugin or Async Workflow?

- Async plugins and workflows both run post-transaction.
-

Execution is not strictly ordered between them.

- - However, in real-world behavior:
 -
 - **Async plugins** often run faster.
 - **Async workflows** are queued in the system job scheduler and may run after a delay.
-

10. A Website Form Is Filled — How to Create a Record in CRM?

Several ways to handle this:

- **Power Pages (Portals):** Use Entity Form Mapping ☒ Automatically creates CRM records.
- **Custom Website:**
 - Use CRM Web API
 - Or trigger **Power Automate flow** via HTTP request

Example:

Form ☒ Web API Call ☒ POST to <https://yourorg.api.crm.dynamics.com/api/data/v9.2/leads>

11. Do Actual Records Get Created When Using FakeXrmEasy in Unit Test Cases?

- No actual records are created.
 - FakeXrmEasy uses an in-memory CRM simulation.
 - It's used for **unit testing** plugins or workflows without needing a CRM instance.
-

12. What Are the Drawbacks of Using FetchXML?

- Limited debugging and logging.
 - Difficult to write and maintain for complex joins.
 - Not flexible for dynamic queries.
 - Lacks advanced filtering (compared to LINQ or QueryExpression).
-

13. What's the Difference Between FetchXML and Fetch API?

Feature	FetchXML	Fetch API (Web API)
Format	XML	JavaScript / JSON

Usage	Views, reports, plugins	Client-side scripts (JS)
Flexibility	Static	Dynamic
Query Type	Complex joins, aggregates	RESTful queries (CRUD operations)

14. Drawbacks of QueryExpression?

- Very verbose syntax in C#.
- Difficult to maintain for deeply nested joins.
- Cannot be reused easily outside SDK (e.g., in views or reports).

15. Pagination Relationship Behavior?

- When using FetchXML or QueryExpression with pagination:
 - Only parent entity records are paged.
 - Related child records (via link-entity) are fetched for those parent records but **not paginated separately**.

Example:

Fetch top 3 Accounts with their Contacts ☒ Only 3 Accounts returned, each with all related Contacts.